Practical - 1

Practical Title: Project Initiation, Requirement Analysis, Sitemap, Wireframe, and GitHub Setup

Problem Definition: Initiate the semester-long StudentHub portal by identifying the problem scope, user roles, key modules, navigation flow, and minimum 10 pages. Create a sitemap, low-fidelity wireframe, project folder structure, README file, and GitHub repository.

A. Theory

1. What is URL and parts of URL?

Answer: A URL (Uniform Resource Locator) is the unique web address used to find and access resources online. Its key components include the scheme, domain name, and path.
Core URL Components
• Scheme (Protocol): Shows the rule or method used to talk to the server, like http or https. It ends with a colon (:).
• Subdomain: An optional subdivision of the main domain, such as www, which comes before the main name.
• Domain Name: The main, human-readable name of the website (like example), followed by a Top-Level Domain extension like .com or .org.
• Port Number: An optional gateway number (like 443 for secure web traffic) separated by a colon (:).
• Path: Points to the exact file, folder, or page on the server, using forward slashes (/).
• Query String (Parameters): Starts with a question mark (?) and passes extra data or search values to the server using key-value pairs separated by ampersands (&).
• Fragment (Hash): Starts with a hash symbol (#) to jump straight to a specific part or section on that web page.

https://www.geeksforgeeks.org/computer-networks/components-of-a-url/

2. How does HTML file process in web browser?

Answer: -

-A web browser processes an HTML file by converting raw network bytes into a visual, interactive webpage through a sequence called the Critical Rendering Path. This process is handled by the browser's rendering engine (like Blink in Chrome or Gecko in Firefox).
1. Document Loading and Byte Conversion
The browser requests a URL, and the server responds by sending the HTML file as a stream of raw binary bytes. The browser immediately translates these raw bytes into human-readable text characters based on the specified file encoding (typically UTF-8).

2. Tokenization
The browser's HTML parser runs a tokenization algorithm. It breaks down the strings of text characters into distinct structural tokens defined by HTML standards (such as start tags <html>, end tags </html>, attribute names, and text content).
3. Tree Construction (Building the DOM)
The browser converts the tokens into object structures called Nodes. It binds these nodes together into a nested tree structure known as the Document Object Model (DOM) Tree. The DOM tree maps out the exact parent-child hierarchy of the entire document, with <html> serving as the root node.
4. Fetching Resources and Building the CSSOM
As the HTML parser reads the file from top to bottom, it encounters external asset links:
• CSS: When a stylesheet is found, the browser pauses rendering to download and parse the CSS into a CSS Object Model (CSSOM) Tree.
• JavaScript: When a <script> tag is found, it pauses HTML parsing entirely to download and execute the code, because JavaScript has the potential to modify the DOM structure. [1]
• Images/Media: These are fetched asynchronously in the background by a preload scanner, allowing HTML parsing to continue without waiting for the images to download.

5. Creating the Render Tree

Once the DOM and CSSOM structures are ready, the browser combines them to create the Render Tree. This tree only includes the visible elements required to paint the page (elements set to display: none; are omitted). Each node in the render tree contains both its structural data and its calculated style rules.

6. Layout (Reflow)

The browser calculates the exact geometry of the screen. It evaluates the device's viewport size and determines exactly how many pixels each Render Tree element will occupy, along with its precise coordinates on the page.

7. Painting and Compositing

In the final step, the browser converts the calculated layout geometry into actual pixels on the screen. It paints backgrounds, borders, text colors, and images onto different software layers. Finally, it composites these layers together via the GPU to output the final visual webpage to the user's screen.

(https://developer.mozilla.org/en-US/docs/Web/Performance/Guides/How_browsers_work)

3. How will page navigation flow managed among all HTML pages?

Answer: -

<nav> HTML navigation section element

The <nav> HTML element represents a

section of a page whose purpose is to

provide navigation links, either within

the current document or to other

documents. Common examples of navigation sections are menus, tables of contents, and indexes.

Reference : (https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/nav#:~:text=The%20HTML%20element%20represents,tables%20of%20contents%2C%20and%20indexes.)

4. How will GitHub commits be maintained after each practical?

Answer:

- git init
- git add .
- git commit -m “any commit message”
-git push origin main/master

(https://github.com/git-guides)

Implementation/ Output/Result Project

Requirements for StudentHub-Portal

Pages Included:

- index.html
- PAGES/home.html
- PAGES/about.html
- PAGES/admin.html

- PAGES/contact.html

- PAGES/dashboard.html

- PAGES/edit.html

- PAGES/events.html

- PAGES/faq.html

- PAGES/feedback.html

- PAGES/login.html

- PAGES/profile.html

- PAGES/register.html

Core Requirements:

- Static web application using HTML, CSS, and JavaScript

- Responsive layout for desktop and mobile screens

- Navigation across all pages via links or menu

- Login and registration forms on `PAGES/login.html` and `PAGES/register.html`

- User dashboard and profile pages for authenticated users

- Admin interface on `PAGES/admin.html`

- Contact page with message submission form

- Events listing page on `PAGES/events.html`

- FAQ, feedback, and about pages for user support and information

- Edit page to allow profile or content updates

B. Evidence of Work (Screenshots)

SiteMap:

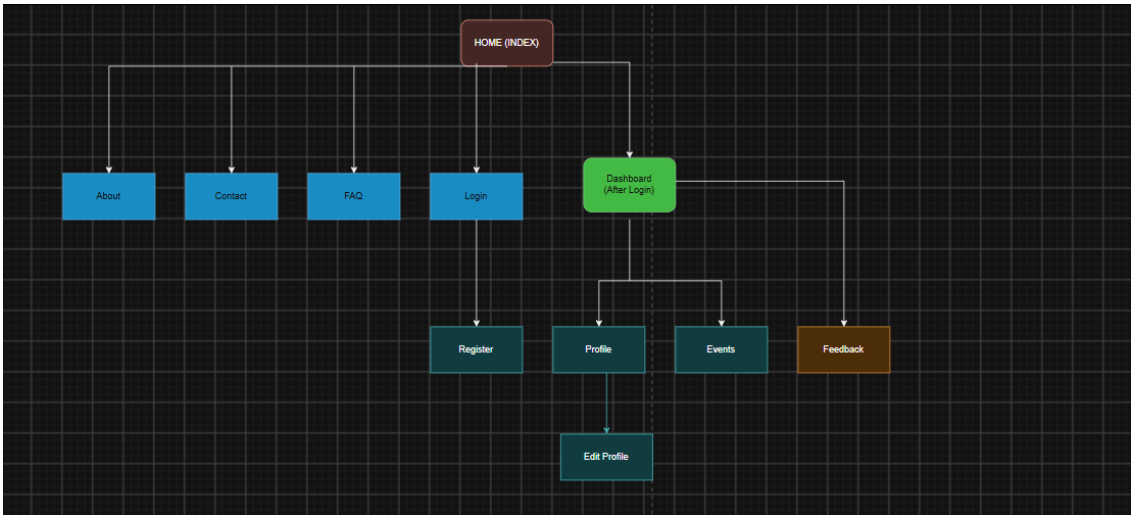


Figure 1.1

WireFrame:

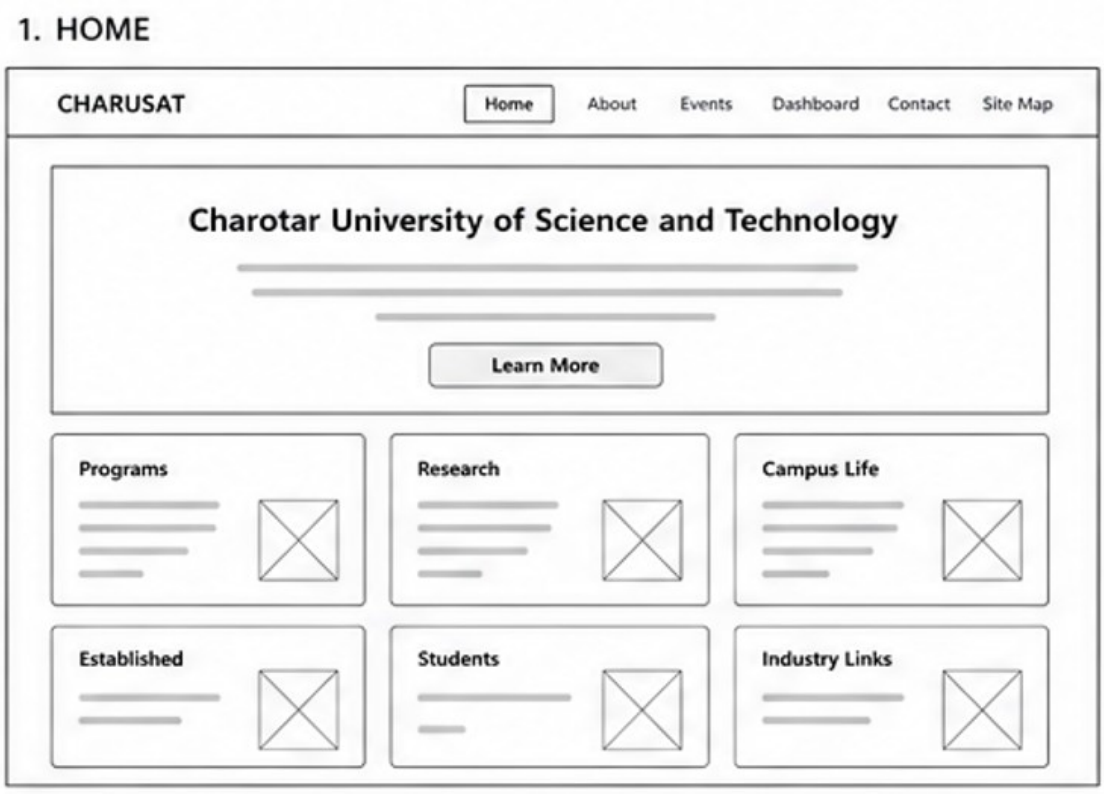


Figure 1.2(Home page)

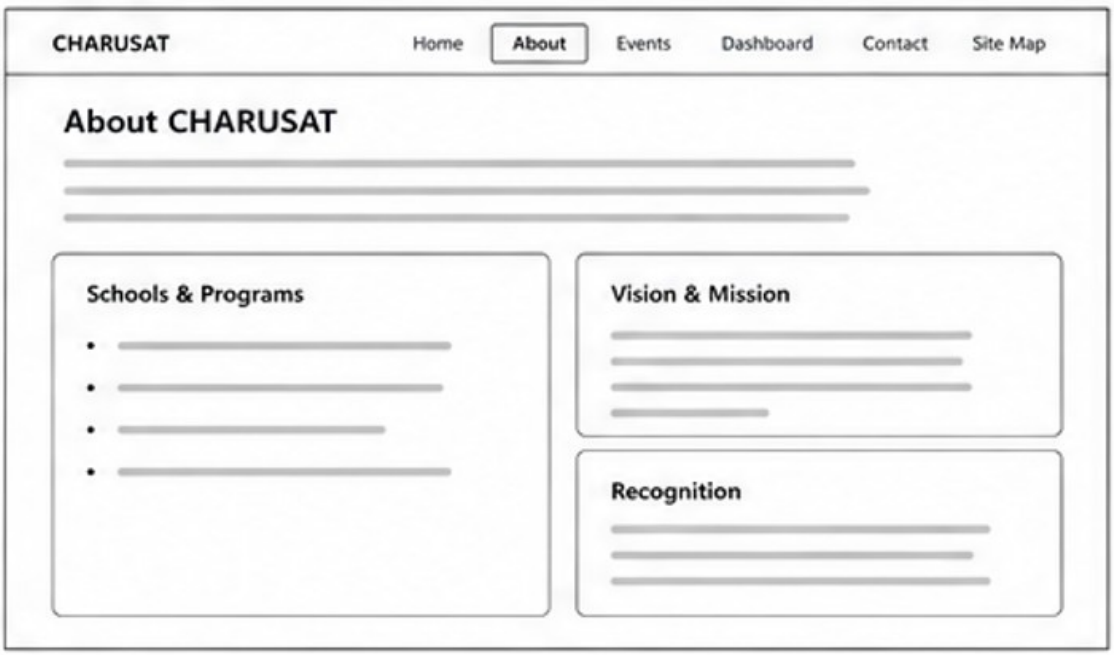


Figure 1.3(About page)



Figure 1.4(Register page)



Figure 1.5(Login page)

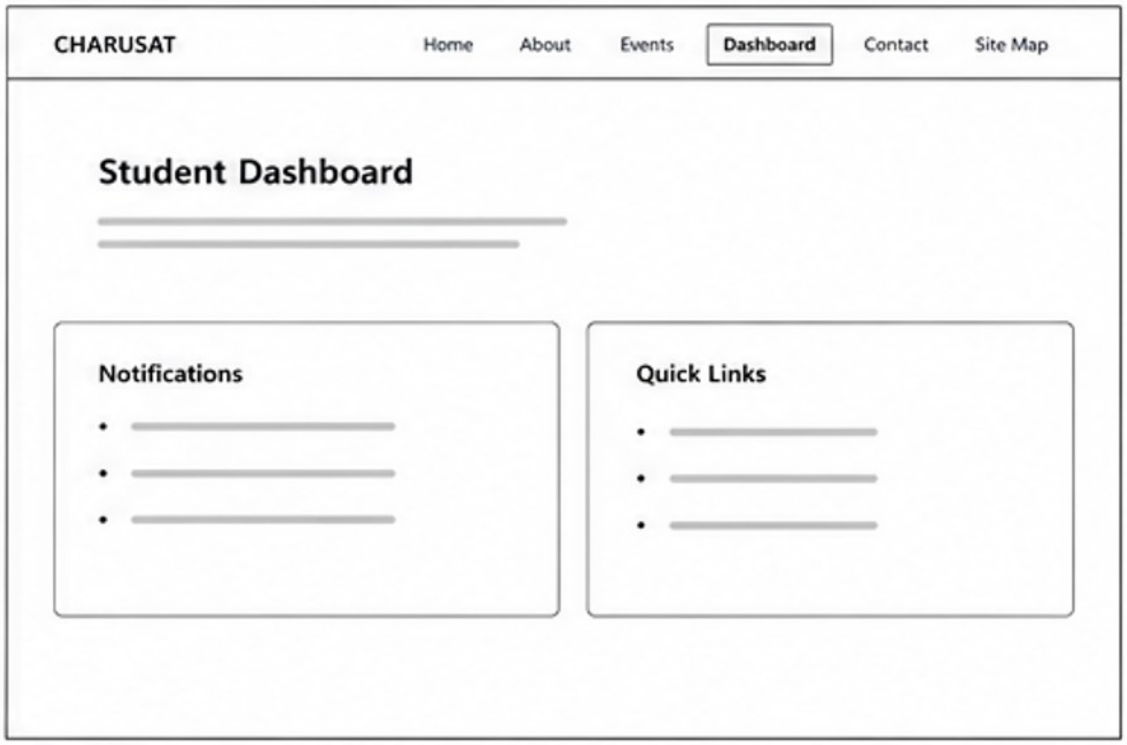


Figure 1.6(Dashboard page)



Figure 1.7(Event page)

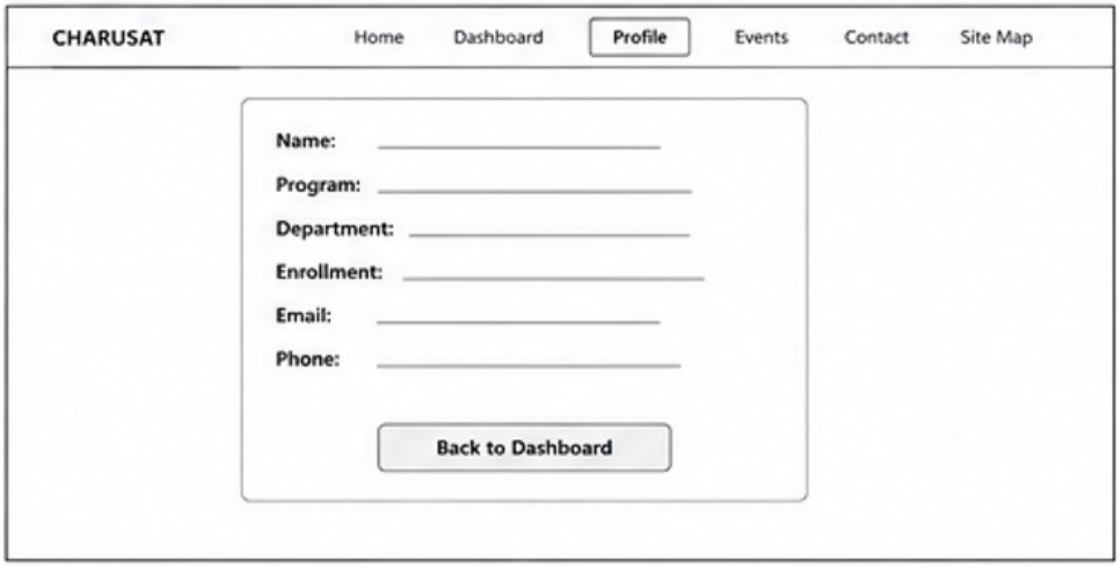


Figure 1.8(Profile page)



Figure 1.10(Contact page)

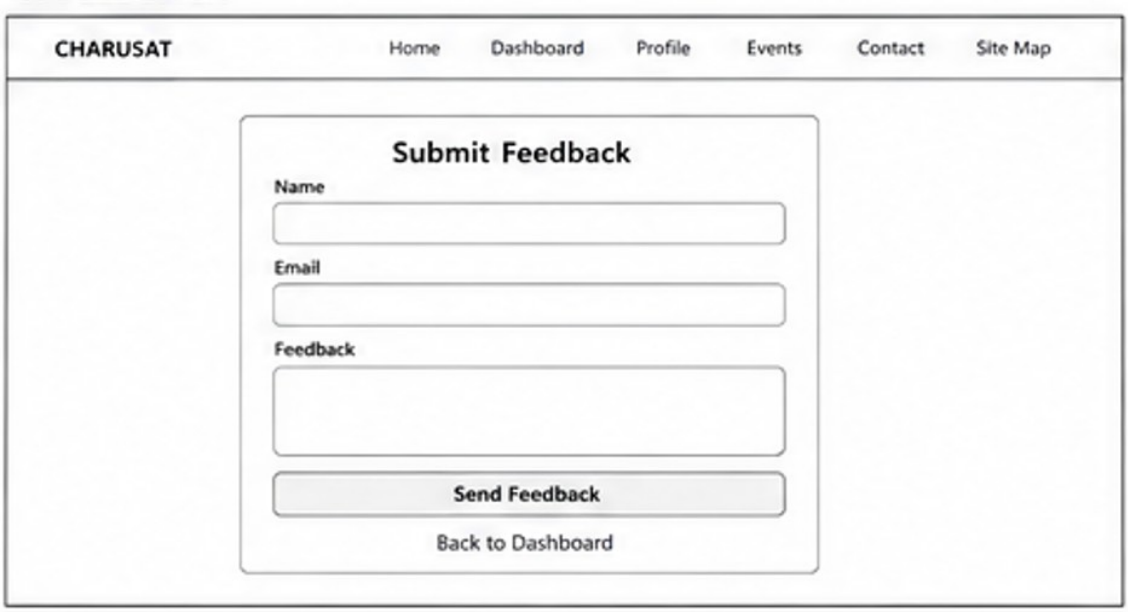


Figure 1.11(Feedback page)

C. GitHub Repository Link:

https://github.com/meet-3834/Student-Hub/tree/main/FINAL%20Submission

D. Challenges Faced:

At start I no idea of sitemap or wireframe or how I will make website in future…then I searched for reference of website structure & asked Ai to give me example of sitemap & wireframe ..with that reference I made sitemap & wireframe in Draw.io . & also started to learn about github push,pull.

E. Learning Outcomes:

Now , I have basic idea About how I can visualize website from scratch by making sitemap & wireframe.

Faculty Remarks: (Keep this blank for faculty)